Convenient Algebraic Programming with Coercive Subtyping

Henry Blanchette[1*] and Aaron Stump[2*]

[1]University of Maryland.
[2]Boston College.

*Corresponding author(s). E-mail(s): blancheh@umd.edu; stumpaa@bc.edu;

Abstract

This paper presents a system for more convenient algebraic programming, where datatypes are defined as least fixed points of covariant signature functors, and recursive functions are defined as algebras. This approach requires rolling and unrolling fixed points, and conversion of algebras to functions that recurse over data. To alleviate the burden of these extra operations, in this paper we propose to insert them automatically, using coercive subtyping. The type checker generates subtyping constraints, which are then solved with the help of a custom logic programming engined called Chronolog. The subtyping axioms are given to Chronolog as rules, which uses them to solve the subtyping constraints. To avoid infinite applications of rules for unrolling signature functors, Chronolog provides a mechanism for suspending and resuming certain forms of goals. The subtyping axioms presented to Chronolog are an equivalent algorithmic version of the usual declarative rules for algebraic programming.

Keywords: strong functional programming, type-based termination

TODO Henry: Make sure use of roll1/roll2 and roll/unroll is consistent
TODO Henry: Cite previous work on handling delayed goals in logic programming

1 Introduction

Strong functional programming is a form of pure functional programming where all functions are statically required to terminate on all inputs. One way to achieve this is through an algebraic approach: datatypes are least fixed points μF of signature functors F, and recursive functions are least fixed points $(\!|a|\!)$, called catamorphisms, of algebras a. The functor F describes one layer of the data, and μF then describes data consisting of any finite number of layers. We may unroll μF to $F\ (\mu F)$, and roll it back again. Algebras interpret the operations declared for one layer of the data, and catamorphisms apply algebras across all the layers. There is no other mechanism for recursion or looping in the language, leading leading to a strong guarantee: all functions written in this style terminate.

One burden is the need to include calls to roll and unroll functors, and to $(\!|-|\!)$, in code. This adds clutter, compared to regular functional programming. In this paper, we address this problem by automatically inserting those coercions wherever they are needed in code. The resulting code that looks indistinguishable from regular pure-functional code, and yet falls within the algebraic style, thus assuring termination. To insert these calls automatically, we turn to *coercive subtyping*. The type checker generates subtyping constraints that need to hold, in order for the term to be well-typed. The constraints are then processed by a constraint solver, generating coercions that are then inserted at the places where the subtyping constraints were generated.

To implement this, we make use of a custom logic-programming engine called Chronolog. The subtyping axioms are given to Chronolog as rules. Chronolog then uses standard SLD resolution to search for proofs of the constraints, instantiating type variables during proof search via unification. The rules for rolling and unrolling signatures would, if handled naively, lead to diverging proof search. Instead, Chronolog provides a mechanism to indicate that certain problematic goals should be suspended. These suspended goals may be resumed if they are instantiated enough during solving of other goals. If not, an outer loop of our algorithm handles them by simply equating the lower and upper bound of the subtyping constraints.

Standard subtyping rules generally are not suitable for logic programming, due to the presence of a transitivity rule, which nondeterministically guesses B with $A <: B$ and $B <: C$, to solve $A <: C$. We identify (Section 6) an algorithmic set of subtyping rules, without transitivity, and prove it equivalent to the declarative rules. This both ensures we can use logic programming (with our algorithmic rules) to solve these subtyping constraints, and also assures us that the algorithmic rules are indeed correct. This proof of equivalence has been formalized and checked in Agda. To summarize the contributions:

- We propose a new approach to total algebraic programming, where calls for rolling and unrolling signature functors and turning algebras into catamorphisms are inferred by coercive subtyping.
- We realize this approach with constraint-generating typing, and a constraint solver based on logic programming, modified to avoid infinite unrolling of functors.
- We propose an algorithmic subtyping system, and prove it equivalent to the declarative one in Agda.

2 Motivating examples

```
sub : μ Nat → Nat ⇒ μ Nat
sub = λ m . ω sub'(n) . γ n .
  { Zero → m
  | Suc(n') → γ coerce (unroll id) (sub' n') .
    { Zero → coerce (roll id) Zero
    | Suc(m') → m' } }

subs : NatList ⇒ μ NatList → μ NatList
subs = ω subs'(l1) . λ l2 . γ l1 .
  { Nil → Nil
  | Cons(h1, t1) → γ coerce (unroll id) l2 .
    { Nil → coerce (roll id) Nil
    | Cons(h2, t2) →
        coerce (roll id)
          Cons((coerce (cata id[μNatList] id[μNatList]) (sub h1)) h2,
                subs' t1 t2) } }
```

Fig. 1: Algebraic implementation of `sub` and `subs`, with explicit coercions.

Figure 1 shows code with explicit coercions, for two functions. The `sub` function subtracts two natural numbers, and `subs` proceeds through a two lists of numbers, subtracting corresponding elements. So subtracting the list (using Haskell syntax) $[1, 2, 3]$ from $[8, 9, 10]$ produces $[7, 7, 7]$, for example. Let us see how these use the different kinds of coercions. The `sub` function takes in the minuend `m` and returns an algebra, which can be used to recurse through the subtrahend `n`. The code uses a γ-term to match on `n`, returning `m` in the `Zero` case. In the `Suc(n')` case, we recursively subtract `n'` (from `m`). This requires unrolling the signature functor with a call to `unroll`. The coercion is actually `unroll id`, where `id` represents that there is no further coercion under this unrolling. The code matches on this result. In the `Zero` case, the code returns `Zero`. Constructors' return types are applications of signature functors. Here, `Zero` can be given type `Nat (`μ `Nat)`, which is then coerced to μ `Nat` by `roll`. The function `subs` does similar rolling and unrolling, but also invokes subtraction (`sub`). Since `sub` returns an algebra, the code uses `cata` to coerce that to the function which accepts the subtrahend.

While these functions have the desirable property that they are guaranteed to terminate on all inputs, we are sure the reader will agree that the code is rather burdened with the need to call these coercions. This raises the question we address in this paper: can we support algebraic programming like this with lighter syntax?

3 Syntax

TODO Henry: Write prose

TODO Aaron: Write prose to introduce syntax of langauge, and explain some details of the running examples from "Motivating Examples".

TODO Henry: Do I actually need to define the domains for term variables, coercion variables, etc. as sets or is there a more standard way?

TODO Henry: Somehow we need to say that type variables can *only* appear in subtyping constraints, since we don't want to claim the system supports polymorphism

$$
\begin{array}{rrcl}
\text{(Datatype Names)} & D & \in & \mathbf{DatatypeNames} \\
\text{(Term variables)} & x & \in & \mathbf{TermVars} \\
\text{(Coercion variables)} & r & \in & \mathbf{CoercionVars} \\
\text{(Recursion Type variables)} & R & \in & \mathbf{RecTypeVars} \\
\text{(Type variables)} & X, Y & \in & \mathbf{TypeVars} \\
\text{(Contexts)} & \Gamma & ::= & \cdot \mid \Gamma, (x : A) \mid \Gamma, (R : *) \\
\text{(Types)} & A, B & ::= & X \mid R \mid \mu D \mid D\ A \mid A \to B \mid A \Rightarrow B \\
\text{(Terms)} & a, b, f & ::= & x \mid D.k(\overline{a,}) \mid \lambda(x : A).b \mid (f\ a) \mid \omega f(x).b \mid \\
 & & & \gamma a.\{\overline{k(\overline{x,}) \mapsto b \mid}\} \mid \text{let } x = a \text{ in } b \mid \text{coerce } c\ a \\
\text{(Coercions)} & c, c_1, c_2 & ::= & r \mid \text{cata } c_1\ c_2 \mid \text{roll } c \mid \text{unroll } c \mid \text{cov}[D]\ c \mid \text{id}[\mu D] \mid \\
 & & & \text{arr } c_1\ c_2 \mid \text{alg}[D]\ c
\end{array}
$$

4 Typing

TODO Henry: Write prose for introducing type system and explain how it fits into the running examples from "Motivating Examples". Here's the idea: Here are the most interesting typing rules. The rest are in Appendix A.

5 Type checking with constraints

We present a type checking procedure based on generating and solving subtyping constraints. First, subtyping constraints are generated from a structurally recursive analysis of a term, where each constraint corresponds to a position in the term at which

$$\frac{\textsc{Gamma}\quad a : D\ C \qquad (\forall (k_i : (\overline{A_{ij}}) \to D\ C) \in \mathscr{K}(D, C))\ \Gamma, \overline{x_{ij} : A_{ij} \vdash b_i : B_i}}{\Gamma \vdash \gamma a.\{\overline{k_i(\overline{x_{ij}}) \mapsto b_i\ |}\} : B}$$

$$\frac{\textsc{Omega}\quad \Gamma, (R : *), (f : R \to A), (x : D\ R) \vdash b : B}{\Gamma \vdash \omega f(x).b : D \Rightarrow B} \qquad \frac{\textsc{Subtype}\quad \Gamma \vdash A <: A' \qquad \Gamma \vdash a : A}{\Gamma \vdash a : A'}$$

Fig. 2: Notable typing rules

a coercion (which may end up being the identity) is required to justify the subtype relation. Then, these subtyping constraints are solved by a logic-programming engine by applying the algorithmic subtyping rules in Figure 6, where each rule corresponds to a constructor of the coercion to be inserted at the constraint's associated position in the term.

TODO Henry: Need to enforce that R and r are linked to this *appearance* of ω. I could add position annotations to all terms and subtype constraints, but that's quite verbose, so I should probably just say this in prose somewhere.

TODO Henry: Is is a problem that r is introduced fresh here even though its not a type variable?

TODO Henry: Enforce exhaustive pattern match at Gamma

Constraint generation for a term a in context Γ is performed by applying the constraint generation rules in Figure 3 recursively on the structure of a, which upon success produces a derivation of $\Gamma \vdash a : A \mid \mathcal{A} \implies \mathcal{C}$. This is a usual typing judgment augmented with a constraint problem $\mathcal{A} \implies \mathcal{C}$, where $\mathcal{A}$ is a set of axioms and $\mathcal{C}$ is a set of goals. Each constraint in these sets is of the form $A <::^p B$, where p is a position of a sub-term of a. The system is designed such that a solution to $\mathcal{A} \implies \mathcal{C}$ specifies an insertion of coercions around sub-terms of a, where each coercion inserted at a position p corresponds to one of the original required constraints $A <::^p B$.

The constraint generation phase also performs type inference, however in contrast to usual type inference and checking procedures, this procedure cannot decide a term to be mal-typed. The constraint generation procedure will generate a constraint problem for any well-formed and well-scoped Γ, a, A. A derivation $\Gamma \vdash a : A \mid \mathcal{A} \implies \mathcal{C}$ merely states that a has type A *only if* the constraint problem $\mathcal{A} \implies \mathcal{C}$ is solvable.

For example, constraint generation for `sub : Nat → Nat ⇒ Nat` results in the following constraint problem, where X, Y, Z, W, U are fresh type variables and R is a fresh recursion type variable:

TODO Henry: Make sure to update this if the constraint generation rules change.

TODO Henry: Insert term positions on constraints

$$\{R <:: R\} \implies \{X \to \mathit{Nat} \Rightarrow U <:: \mu\mathit{Nat} \to \mathit{Nat} \Rightarrow \mu\mathit{Nat}, \mathit{Nat}\ Y <:: \mathit{Nat}\ W, R \to \mu\mathit{Nat} <:: R \to \mathit{Nat}\ Y, \mathit{Nat}\ Z <:: U, \mu\mathit{Nat} <:: U\}$$

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A \mid \varnothing \implies \varnothing}\ \textsc{Var}$$

$$\frac{X \text{ fresh} \qquad (k : (\overline{B_i,}) \to D\ X) \in \mathscr{K}(D, X) \qquad (\forall i)\ \Gamma \vdash a_i^{p_i} : A_i \mid \mathcal{A}_{a_i} \implies \mathcal{C}_{a_i}}{\Gamma \vdash k(\overline{a_i,}) : D\ X \mid \bigcup_i (\{A_i <::^{p_i} B_i\} \cup \mathcal{A}_{a_i}) \implies \bigcup_i \mathcal{C}_{a_i}}\ \textsc{Con}$$

$$\frac{X \text{ fresh} \qquad \Gamma, (x : X) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \lambda x.b : X \to B \mid \mathcal{A}_b \implies \mathcal{C}_b}\ \textsc{Lam}$$

$$\frac{X \text{ fresh} \qquad \Gamma \vdash f : B \mid \mathcal{A}_f \implies \mathcal{C}_f \qquad \Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a}{\Gamma \vdash f\ a : X \mid \{B <:: A \to X\} \cup \mathcal{A}_f \cup \mathcal{A}_a \implies \mathcal{C}_f \cup \mathcal{C}_a}\ \textsc{App}$$

$$\frac{\begin{array}{cc} X, Y \text{ fresh} & \Gamma \vdash a : C \mid \mathcal{A}_a \implies \mathcal{C}_a \\ (\forall i)\ k_i : (\overline{A_{ij},}) \to D\ X & (\forall i)\ \Gamma, \overline{x_{ij} : A_{ij},} \vdash b_i : B_i \mid \mathcal{A}_{b_i} \implies \mathcal{C}_{b_i} \end{array}}{\Gamma \vdash \gamma a.\{\overline{k_i(\overline{x_{ij},}) \mapsto b_i \mid}\} : Y \mid \mathcal{A}_a \cup \bigcup_i \mathcal{A}_{b_i} \implies \{C <:: D\ X\} \cup \mathcal{C}_a \cup \bigcup_i (\{B_i <:: Y\} \cup \mathcal{C}_{b_i})}\ \textsc{Gamma}$$

$$\frac{R, r \text{ fresh} \qquad \Gamma, (R : *), (r : R <:: R), (f : R \to A), (x : D\ R) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \omega f(x).b : D \Rightarrow B \mid \mathcal{A}_b \cup \{R <:: R\} \implies \mathcal{C}_b}\ \textsc{Omega}$$

$$\frac{\Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a \qquad \Gamma, (x : A) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \text{let } x = a \text{ in } b : B \mid \mathcal{A}_a \cup \mathcal{A}_b \implies \mathcal{C}_a \cup \mathcal{C}_a}\ \textsc{Let}$$

Fig. 3: Constraint generation rules.

TODO Henry: Standardize choice of terminology: "axioms" and "requirements" Here is where the axioms and requirements are introduced.

- The axiom $R <:: R$ is introduced by $\omega\ sub'(n)$.
- The requirement $X \to Nat \Rightarrow U <:: \mu Nat \to Nat \Rightarrow \mu Nat$ is introduced by $\gamma\ n$.
- **TODO** explain sources of other requirements

TODO Henry: Explain anything else necessary for understanding constraint generation

TODO Henry: State completeness and soundness for the constraint generation rules relative to the typing rules

$$\textsc{VarCoe}\quad \frac{(r : A <:: B) \in \Gamma}{\Gamma \vdash r : A <:: B} \qquad \textsc{ReflCoe}\quad \frac{D \text{ data}}{\Gamma \vdash \mathrm{id}[\mu D] : \mu D <:: \mu D}$$

$$\textsc{ArrCoe}\quad \frac{\Gamma \vdash c_1 : A' <:: A \qquad \Gamma \vdash c_2 : B <:: B'}{\Gamma \vdash \mathrm{arr}\ c_1\ c_2 : A \to B <:: A' \to B'} \qquad \textsc{CovCoe}\quad \frac{\Gamma \vdash c : A <:: A'}{\Gamma \vdash \mathrm{cov}[D]\ c : D\ A <:: D\ A'}$$

$$\textsc{AlgCoe}\quad \frac{\Gamma \vdash c : A <:: A'}{\Gamma \vdash \mathrm{alg}[D]\ c : D \Rightarrow A <:: D \Rightarrow A'} \qquad \textsc{RollCoe}\quad \frac{D \text{ data} \qquad \Gamma \vdash c : A <:: \mu D}{\Gamma \vdash \mathrm{roll}\ c : D\ A <:: \mu D}$$

$$\textsc{UnrollCoe}\quad \frac{D \text{ data} \qquad \Gamma \vdash c : \mu D <:: A}{\Gamma \vdash \mathrm{unroll}\ c : D\ A <:: \mu D} \qquad \textsc{CataCoe}\quad \frac{\Gamma \vdash c_1 : B <:: D \qquad \Gamma \vdash c_2 : A <:: C}{\Gamma \vdash \mathrm{cata}\ c_1\ c_2 : D \Rightarrow A <:: B \to C}$$

Fig. 4: Coercion typing rules.

6 Subtyping

We begin with the declarative subtyping rules corresponding to the coercions of Figure 4, and then propose an algorithmic version, where all meta-variables in the premises of the rule are already present in the conclusion, and the sizes of the constraints decrease. This ensures that SLD resolution will not diverge on ground constraints. We prove that the two sets of rules are equivalent.

6.1 Declarative subtyping

$$\textsc{Refl}\quad \frac{D \text{ data}}{\mu D <: \mu D} \qquad \textsc{Arr}\quad \frac{A' <: A \quad B <: B'}{A \to B <: A' \to B'} \qquad \textsc{Cov}\quad \frac{A <: A'}{D\ A <: D\ A'} \qquad \textsc{Alg}\quad \frac{A <: A'}{D \Rightarrow A <: D \Rightarrow A'}$$

$$\textsc{Roll}\quad \frac{}{D\ \mu D <: \mu D} \qquad \textsc{Unroll}\quad \frac{}{\mu D <: D\ \mu D} \qquad \textsc{Cata}\quad \frac{}{D \Rightarrow A <: \mu D \to A} \qquad \textsc{Trans}\quad \frac{A <: A' \quad A' < A''}{A <: A''}$$

Fig. 5: Declarative subtyping rules.

Figure 5 shows the rules for declarative subtyping $A <: B$. We have reflexivity rule Refl for datatypes. Arr is the usual subtyping rule for function types, which is contravariant in the domain and covariant in the codomain. Cov expresses that the signature functor is indeed a (covariant) functor. ALG expresses that algebra types $D \Rightarrow A$ are covariant in their carriers A. ROLL and UNROLL together express that μF is equivalent to $F\ (\mu F)$. CATA expresses that an algebra can be coerced to a function, namely the algebra's catamorphism. Finally, there is the TRANS rule, completing the definition of subtyping as a partial order.

6.2 Algorithmic subtyping

$$\textsc{Refl}\ \frac{D \text{ data}}{\mu D <:: \mu D} \qquad \textsc{Arr}\ \frac{A' <:: A \quad B <:: B'}{A \to B <:: A' \to B'} \qquad \textsc{Cov}\ \frac{A <:: A'}{D\ A <:: D\ A'}$$

$$\textsc{Alg}\ \frac{A <:: A'}{D \Rightarrow A <:: \mu D \Rightarrow A'} \qquad \textsc{Roll}\ \frac{A <:: \mu D}{D\ A <:: \mu D} \qquad \textsc{Unroll}\ \frac{\mu D <:: A}{\mu D <:: D\ A} \qquad \textsc{Cata}\ \frac{B <:: \mu D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

Fig. 6: Algorithmic subtyping rules.

Figure 6 presents the rules for algorithmic subtyping $A <:: B$. We use similar names as for the declarative rules, but critically, the algorithmically problematic TRANS rule is not present. A consequence of this is that now several rules that did not have premises in the declarative formulation, here do. For example, compare the declarative (on the left) and algorithmic CATA rules:

$$\frac{}{D \Rightarrow A <: \mu D \to A} \qquad \frac{B <:: \mu D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

The algorithmic rule needs to take into account subtyping relationships between the components in the conclusion, because there is no rule for transitivity, which could allow those relationships to be taken into account after this rule is applied. In effect, transitivity needs to be built into all the other rules. We see a similar change from the declarative to the algorithmic versions of the ROLL and UNROLL rules, for rolling and unrolling the signature functor.

Lemma 1 (Decrease) *For all rules of Figure 6, the sizes of the constraints in the premises are less than the size of the constraint in the conclusion.*

Theorem 2 (Decidability) *It is decidable whether or not two types are in the algorithmic subtyping relation.*

Proof Bottom-up application of the rules to ground types is guaranteed to terminate, by Lemma 1. Therefore, proof search may be used as a decision procedure for algorithmic subtyping of ground types. □

6.3 Equivalence of declarative and algorithmic subtyping

While Theorem 2 ensures that we can test ground subtyping constraints using proof search, we must confirm that the algorithmic rules are indeed a correct version of the declarative ones. For this, we prove the following theorems.

Theorem 3 (Soundness) *$A <:: B$ implies $A <: B$.*

Theorem 4 (Completeness) *$A <: B$ implies $A <:: B$.*

The sets of rules are quite similar, so these proofs are not onerous. They do depend on the following inductively proven lemmas.

Lemma 5 (roll1 (declarative)) *If $A <: \mu D$, then $D\ A <: \mu D$.*

Lemma 6 (roll2 (declarative)) *If $\mu D <: A$, then $\mu D <: D\ A$.*

Lemma 7 (refl (algorithmic)) *$A <:: A$.*

Lemma 8 (trans (algorithmic)) *If $A <:: B$ and $B <:: C$ then $A <:: C$.*

Lemma 9 (roll (algorithmic)) *$D\ \mu D <: \mu D$*

Lemma 10 (unroll (algorithmic)) *$\mu\ D <: D\ \mu D$*

6.4 Formalization in Agda

Types are represented by the Agda datatype `Ty` in Figure 7. For simplicity, we have single signature functor `D` and its least fixed-point `μD` (note that this is a single identifier in Agda, not an application of a μ operator to `D`). Algebra types $D \Rightarrow A$ are represented just as `D⇒ A`. Note that `D⇒` is also a single symbol in Agda, like `μD`. Arrow types are represented $A \longrightarrow B$ (as other arrow notations are already in use). The figure declares some fixity information, for lighter notation where these operators are used later.

Figure 8 show the representation of the algorithmic subtyping system; we omit a similar definition for declarative subtyping, for space reasons. Each set of rules is represented as an indexed inductive type. Each rule becomes a constructor of the type. For example, we have, for both types, `Refl` of type `μD <: μD`, which corresponds exactly to the `Refl` rules of Figures 5 and 6. Conveniently, Agda allows reusing constructor

```
data Tp : Set where
  D_ : Tp → Tp
  _⟶_ : Tp → Tp → Tp
  D⇒ : Tp → Tp
  μD : Tp

infixr 9 _⟶_
infixr 10 D_
```

Fig. 7: Agda definition of types.

```
infix 5 _<::_

data _<::_ : Tp → Tp → Set where
  Refl : μD <:: μD
  Arr : {T1 T2 T1' T2' : Tp} →
         T1' <:: T1 →
         T2 <:: T2' →
         T1 ⟶ T2 <:: T1' ⟶ T2'
  Roll : {T : Tp} → T <:: μD → D T <:: μD
  Unroll : {T : Tp} → μD <:: T → μD <:: D T
  Cov : {T1 T2 : Tp} → T1 <:: T2 → D T1 <:: D T2
  Alg : {T1 T2 : Tp} → T1 <:: T2 → D⇒ T1 <:: D⇒ T2
  Cata : {T T1 T2 : Tp} →
        T1 <:: μD →
        T <:: T2 →
        D⇒ T <:: T1 ⟶ T2
```

Fig. 8: Agda definition of algorithmic subtyping.

names across datatypes. Dependent typing is used for quantification. For example, the `Roll` constructor has type

```
{T : Tp} → T <:: μD → D T <:: μD
```

This is a dependent function type, stating that give `T : Tp` and a proof of `T <:: μD`, `Roll` returns a proof of `D T <:: μD`. Finally, the statements of soundness and completeness are shown in Figure 9. The total number of lines for all the Agda code is under 150. This shows that the proof burden of the approach is very manageable. Part of the reason for this is that the types do not contain bound variables, and so the technical overhead associated with reasoning in the presence of those does not arise.

```
Soundness    : {T T' : Tp} → T <:: T' → T <: T'
Completeness : {T T' : Tp} → T <: T'  → T <:: T'
```

Fig. 9: Agda statements of soundness and completeness.

6.5 Constraint solving with Chronolog

We take a logic-programming approach to solving problems $\mathcal{A} \implies \mathcal{C}$ in such a way that calculates the coercions that justify the generated subtyping relations. Our implementation uses a logic-programming engine, Chronolog, which adopts an alternative technique for goal suspension compared to rule-level techniques in Prolog variants (e.g. Prolog II). **TODO** source Chronolog accepts a set of axioms, a set of goals, a logic variable substitution, and a suspension meta-predicate S. Anytime a goal that satisfies S appears, that goal is immediately suspended. Suspended goals are still refined by logic variable substitutions. Chronolog can terminate in three ways: (1) failure with unsolved goals, (2) progress with suspended goals, (3) success with no suspended goals.

A particular choice of S is critical in this context due to the presence of the Roll and Unroll rules, which can be composed infinitely by straightforward SLD resolution, violating completeness of type checking. Chronolog prevents such infinite regresses by supporting a global suspension meta-predicate, S, where goals that satisfy S are immediately suspended before being processed further. If such a refinement results in a goal no longer satisfying S, then the goal is resumed. We use a meta-predicate for S that holds of these forms of goals, where X, Y are type variables, D is a datatype name, R is a recursion type variable, and A, B are types.

$$X <:: \mu D, X <:: D\ A, X <:: R, \mu D <:: X, D\ A <:: X, R <:: X, X <:: Y$$

Suspending goal of these forms prevents Chronolog from applying rules in a cycle, which follows from a case analysis of the all possible rule applications to goals not of suspended forms. In particular, the only forms of goals that have a type variable as one side of the relation that are not suspended are those were the other side of the relation is either $D \Rightarrow A$ or $A \rightarrow B$. This allows Chronolog, for example, to attempt to solve goals of the form $X <:: A \rightarrow B$ with both Arr and Cata, or to attempt to solve goals of the form $A \Rightarrow B <:: X$ with both Alg and Cata.

Constraints solving is accomplished by using Chronolog inside a loop that processes any suspended goals left over after each iteration, as shown in Algorithm 1. Each iteration, a single suspended goal is chosen to be processed by Refine, resulting in a substitution that refines that suspended goal (and may incidentally refine other suspended goals). The choice of which suspended goal to process may affect the order in which Chronolog explores solutions but all choices lead to equivalent solutions up to permutations of Roll and Unroll (if there is a solution). **TODO** Henry: justify this statement

Algorithm 1 Constraint solving with Chronolog.

function $\textsc{Solve}(axioms, goals)$
  $\sigma \leftarrow \emptyset$
  while $goals \neq \emptyset$ **do**
    $result \leftarrow \textsc{Chronolog}(S, axioms, goals, \sigma)$
    match $result$ **with**
      case $\langle$"failure"$\rangle \Rightarrow$ **return** $\bot$
      case $\langle$"success"$, \sigma'\rangle \Rightarrow$ **return** σ'
      case $\langle$"progress"$, \sigma', goals'\rangle \Rightarrow$
        $goal \leftarrow \textsc{Sample}(goals')$
        $\sigma \leftarrow \textsc{Refine}(goal) \circ \sigma'$
        $goals \leftarrow \sigma(goals')$
  end while
end function

$$
\begin{aligned}
\textsc{Refine}(\mu D <:: X) &= \textsc{Refine}(X <:: \mu D) &&= \{X \mapsto \mu D\} \\
\textsc{Refine}(D\ A <:: X) &= \textsc{Refine}(X <:: D\ A) &&= \{X \mapsto D\ Y\} \text{ where } Y \text{ fresh} \\
\textsc{Refine}(R <:: X) &= \textsc{Refine}(X <:: R) &&= \{X \mapsto R\} \\
&\ \textsc{Refine}(X <:: Y) &&= \{X \mapsto Y\}
\end{aligned}
$$

Theorem 11 (Termination) Solve *terminates.*

Proof We show termination by demonstrating a strictly decreasing metric over each iteration of the loop in Solve. Each iteration begins with an invocation of Chronolog. If Chronolog results in ⟨"failure"⟩ or ⟨"success"⟩, then we are done. If Chronolog results in ⟨"progress", σ', *goals*$'$⟩, then by Lemma 1 the leftover *goals*$'$ must be in total smaller than the original *goals* from the beginning of this iteration.

The substitution resulting from Refine(*goal*) preserves the size of goals in all but one case. When Refine(*goal*) increases the size of goals in one case, where it results in $\{X \mapsto D\ Y\}$, Chronolog will strictly decrease the size of these goals again immediately, as follows from a case analysis on all the possible results of applying Refine(*goal*) to suspended goals. □

Theorem 12 (Completeness) Solver *is a decision procedure for constraint problems* $\mathcal{A} \implies \mathcal{C}$.

Proof Solve will finds all solutions (up to permutations of Roll and Unroll) to $\mathcal{A} \implies \mathcal{C}$ because Chronolog attempts every valid application of the algorithmic subtyping rules which are complete by Theorem 4, Refine preserves validity of solution σ, and Solve terminates by theorem 11. □

7 Returning to the Examples

TODO Henry: Prose. Remind the version with coercions. Show without coercions

```
sub : μ Nat → Nat ⇒ μ Nat
sub = λ m . ω sub'(n) . γ n .
  { Zero → m
  | Suc(n') → γ sub' n' .
    { Zero → Zero
    | Suc(m') → m' } }

subs : NatList ⇒ μ NatList → μ NatList
subs = ω subs'(l1) . λ l2 . γ l1 .
  { Nil → Nil
  | Cons(h1, t1) → γ l2 .
    { Nil → Nil
    | Cons(h2, t2) →
        Cons(sub h1 h2, subs' t1 t2) } }
```

Fig. 10: Algebraic implementation of `sub` and `subs`.

8 Related Work

Turner introduced the term *strong functional programming*, presented several arguments in favor of the approach, and proposed using structural termination as an *elementary* way to realize such a language [1]. Mendler proposed a recursor using an abstract type to ensure that recursive calls are permitted only on subdata for an inductive type [2]. The algebraic approach has been applied for modular implementation of interpreters [3]. Charity is a strong functional programming language based on categorical abstractions for initial algebras and final coalgebras [4]. Tofu uses the Calculus of Constructions as a language for defining terminating functions, again based on categorical ideas [5]. Charity and Tofu both support coinductive datatypes, which we did not study here.

9 Conclusion and future work

Total algebraic programming can be made more feasible by inferring the basic coercions necessary for the method: rolling and unrolling signature functors, and turning an algebra into its catamorphism. Type inference generates subtyping constraints, whose solutions correspond to the needed coercions. A logic-programming engine processes constraints with respect to an algorithmic version of the subtyping rules. This algorithmic subtyping relation is proved equivalent to the declarative one, in Agda.

Future work includes richer forms of algebraic programming. One powerful generalization of structural recursion is to recursive coalgebras [6]. This encompasses divide-and-conquer algorithms, which are beyond the scope of structural recursion. A different approach to divide-and-conquer recursion has also been considered by Abreu et al. [7]. There, algebras call functions to pass between various abstract types. These would be excellent candidates for inference using coercive subtyping, as we proposed in this paper. It would also be interesting to infer coercions for coalgebraic programming with coinductive types.

A Typing rules

TODO Henry: full typing rules

References

[1] Turner, D.A.: Elementary Strong Functional Programming. In: Proceedings of the First International Symposium on Functional Programming Languages in Education. FPLE '95, pp. 1–13. Springer, Berlin, Heidelberg (1995)

[2] Mendler, N.P.: Inductive types and type constraints in the second-order lambda calculus. Annals of Pure and Applied Logic **51**(1), 159–172 (1991)

[3] Swierstra, W.: Data Types à La Carte. J. Funct. Program. **18**(4), 423–436 (2008)

[4] Cockett, R., Fukushima, T.: About Charity. Yellow Series Report No. 92/480/18, Department of Computer Science, The University of Calgary (June 1992)

[5] Widemann, B.T.: Tofu - towards practical church-style total functional programming. In: Workshop der GI-Fachgruppe Programmiersprachen und Rechenkonzepte, Bad Honnef, 3.-5. Mai (2010). Available at https://www-ps.informatik.uni-kiel.de/fg214/Honnef2010/TechnischerBericht1010.pdf

[6] Capretta, V., Uustalu, T., Vene, V.: Recursive coalgebras from comonads. Inf. Comput. **204**(4), 437–468 (2006) https://doi.org/10.1016/J.IC.2005.08.005

[7] Abreu, P., Delaware, B., Hubers, A., Jenkins, C., Morris, J.G., Stump, A.: A type-based approach to divide-and-conquer recursion in coq. Proc. ACM Program. Lang. **7**(POPL), 61–90 (2023) https://doi.org/10.1145/3571196